

Your Paper Title

Anonymous Author(s)

Abstract

LLM agent frameworks increasingly expose model invocations through structured programming abstractions, with framework runtimes lowering program-level constructs into concrete prompts. We observe that these programmatically generated prompts are inherently heterogeneous: their components exhibit different sharing scopes, lifetimes, and evolution patterns across invocations. However, existing serving systems treat prompts as opaque text, losing this structure and missing opportunities for more effective KV-cache reuse and management.

We present IDONTKNOWITSNAME, a serving layer that recovers and exploits prompt heterogeneity for efficient KV-cache management. IDONTKNOWITSNAME lifts incoming prompts against framework templates to recover call sites and binding boundaries, learns binding evolution patterns and application-level control flow from execution traces, and uses this information to optimize KV-cache behavior at two levels. Within a request, IDONTKNOWITSNAME reorders bindings by lifetime to expose stable, reusable prefixes. Across requests, a lifecycle-aware planner proactively prefetches prefixes likely to be reused and evicts cache entries with low future utility. We implement IDONTKNOWITSNAME on SGLang and evaluate it against multiple established serving systems. IDONTKNOWITSNAME achieves up to $1.4\times$ – $1.7\times$ lower mean JCT and $1.7\times$ – $1.75\times$ lower mean TTFT than vanilla SGLang.

1 Introduction

Agent frameworks increasingly expose LLM calls as programming abstractions rather than manual prompt construction. ByLLM [3] introduces a language-level abstraction that delegates function execution to an LLM, with the runtime using semantic information from the code like function signatures and parameter names to construct model requests. NVIDIA Object-Oriented Agents (NOOA) [5] similarly embeds LLM-driven behavior into native Python objects and translates program-level declarations into prompts and model interactions. DSPy [7] represents LM invocations through declarative signatures and modules, while the framework automatically generates a concrete prompt. Across these systems, developers specify LLM behavior using structured programming constructs, while framework runtime translate those constructs into concrete prompts and model requests.

This trend extends beyond hand-written agent code. SIGIL [4] compiles natural-language skill documents into programs with explicit control flow and typed LLM calls, which the runtime lowers into prompts. Increasingly, requests arriving at inference engines encode structure inherited from

the programs that generate them. Existing serving systems, however, make limited use of this structure. Engines such as SGLang [13] exploit exact token-prefix matches for KV-cache reuse without recognizing the program components within a prompt. Recent agent-serving systems, including PBKV [12], CacheScout [11], KVFlow [9], and Pythia [10], exploit workflow information to anticipate future invocations and guide cache management, but largely overlook the information encoded in the prompt itself. As a result, these systems fail to fully leverage the structure and semantics of programmatically generated prompts, leading to suboptimal cache management and performance.

We argue that these programmatically generated LLM requests are inherently heterogeneous. Components within a generated prompt can have different sharing scopes and lifetimes. For example, some components may remain fixed across invocations, some are shared across call sites, and some can grow or be replaced as execution progresses. As the instance in figure 1 shows, the four bindings of `assess_evidence` have distinct lifetimes and evolution patterns. A developer may naturally place `search_results` and `question` before evidences in the function signature, and the runtime preserves this ordering when lowering the invocation into a prompt. However, existing serving frameworks treat the requests as opaque text. Once requests reach the serving system, the original program structure is no longer visible, creating a gap between the application level runtime and the serving layer. Serving engine blindly consumes the prompt in such order means any changes to either binding prevent prefix-cache reuse of unchanged entries in evidences. Additionally, because the serving layer holds no knowledge about the binding heterogeneity, it fails to identify which kvcache is evictable, which leads to a suboptimal management.

To close this gap, we propose IDONTKNOWITSNAME, a serving layer that exploits prompt heterogeneity to achieve more efficient kvcache management. IDONTKNOWITSNAME recovers each request’s call site and binding boundaries by lifting the prompt text against the framework’s templates, learns heterogeneous binding evolution pattern and application-level structure from traces, and then conducts progressive kvcache management. IDONTKNOWITSNAME exploits such heterogeneity at two levels: within a request, it reorders bindings by lifetime so that stable content forms a shareable prefix; across requests, a lifecycle-aware planner uses the learned control flow to prefetch prefixes before they are needed and evicts those that won’t be reused. We implemented IDONTKNOWITSNAME based on SGLang [13]. Our evaluations shows that IDONTKNOWITSNAME achieved up to $1.4\times$ – $1.7\times$ lower mean JCT and $1.7\times$ – $1.75\times$ lower mean

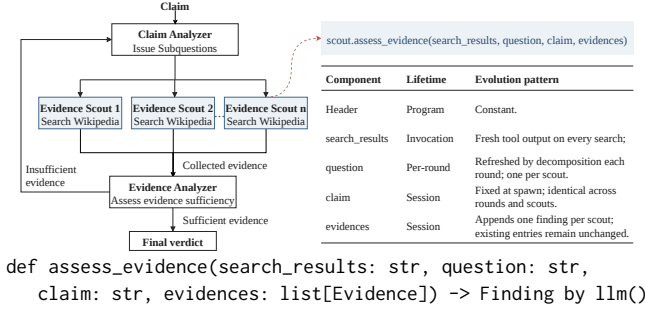


Figure 1. Heterogeneity within a HoVer-style [6] Fact Check Agent’s generated prompt for the summary making function. `assess_evidence(search_results, question, claim, evidences)` evaluates the sub-question against accumulated evidences and search results, then gives answer to it.

TTFT than vanilla SGLang. IDONTKNOWITSNAME also outperforms multiple established serving systems.

In summary, we made the following contributions.

- We identified the inherent heterogeneity in programmatically generated prompt and its implications for serving.
- We designed and built IDONTKNOWITSNAME, a serving layer that exploit prompt heterogeneity to guide more efficient kvcache scheduling.
- We conducted comprehensive evaluations on our IDONTKNOWITSNAME against multiple established serving systems, demonstrating that IDONTKNOWITSNAME achieves up to $1.4\times$ – $1.7\times$ lower mean JCT and $1.7\times$ – $1.75\times$ lower mean TTFT than vanilla SGLang.

2 Background

Agent frameworks expose LLM calls through programming abstractions and translate them into prompts at runtime, but existing agent serving frameworks ignore the optimization opportunity carried by that. A LLM call declared in that way

3 Design

We designed IDONTKNOWITSNAME, a serving layer that exploits the heterogeneity of programmatically generated LLM requests to schedule the KV cache more efficiently, closing the gap between the runtime that generates prompts and the engine that serves them.

IDONTKNOWITSNAME employs 2 planes to comprehensively discover and exploit such heterogeneity. The discovery plane first lifts each incoming prompt back to its call-site signature and bindings, then combines this metadata with request history to learn how each binding evolves and the structure of agentic application. The scheduling plane consumes this knowledge at two levels: it reorders out prompts to enlarge prefix reuse, and it places KV segments across

device and host with heterogeneity aware eviction policy to ensure reusable kvcache segments reside in correct tier.

3.1 Discovering Heterogeneity

IDONTKNOWITSNAME first lifts each received prompt back to its call-site metadata. It then analyzes how each binding evolves across the request history along two dimensions, intra-call-site evolution and cross-call-site sharing, and assigns every binding a lifetime and sharing class. Separately, IDONTKNOWITSNAME learns the application’s call structure as a time-annotated probabilistic finite-state machine (TPFM). The result is a per-request record that annotates each binding with its class and future reuse estimator; the scheduling plane then consumes this knowledge.

3.1.1 Lifting prompt back into metadata. The lowering process is lost when prompts crossing the boundary of runtime and serving engine. However, because prompts are lowered by runtime with deterministic processes, these generated requests still retain recoverable structure. Runtime emits a fixed skeleton from the declaration, the header with its signature, schema rows and output-format tail, and drops each bound argument into a slot of that skeleton, labelled by the parameter name and placed in declaration order. Then the skeleton stays statics and only binding values would change.

Lifting is therefore template application that run backwards. We represent a call site x by a prompt template $T_x = \langle S_x, (g_1, \dots, g_m), \tau_x \rangle$, where S_x is the static system prompt, τ_x is a request-only tail, and each segment is either a static byte string or a named slot:

$$g_i ::= \text{STATIC}(b) \mid \text{SLOT}(n, \ell),$$

where n is the parameter name and ℓ is its literal label. Rendering replaces every slot with its binding value, while splitting matches the static segments and labels to recover the bindings. A valid template must satisfy

$$\text{RENDER}(T_x, \text{SPLIT}(T_x, u)) = u$$

for every observed request u of the site.

The compiler may provide T_x directly; otherwise, the serving engine deduces it from call history. Requests are first grouped by a call-site fingerprint consisting of the model, system prompt, and call-message header. Given at least two distinct messages from one site, the engine computes their longest common subsequence of lines. Common lines form the static skeleton and the regions between them are candidate slots. Within each region, repeated labels identify parameter names and split the region into named slots. If no reliable label pattern exists, the complete region remains one conservative slot. A suffix found only in the live request is recorded separately as the request-only tail. The metadata for a programmatically generated prompt typically includes binding’s name and values, system prompt and invariant headers, as shown in 1. Such metadata is obtained by applying template on received prompt.

Algorithm 1 Deducing a prompt template**Require:** call history H_x **Ensure:** T_x or $\perp$

-
- ```

1: $(U, \tau) \leftarrow \text{NORMALIZE}(H_x)$ $\triangleright$ Extract calls and Strip
 request-only tail for distinct messages
2: $K \leftarrow \text{LCS}(\{\text{LINES}(u) : u \in U\})$ $\triangleright$ find recurring lines
3: $R \leftarrow \text{GAPS}(\text{ALIGN}(K, U))$ $\triangleright$ collect changing regions
4: $G \leftarrow \text{SEGMENT}(K, R)$ $\triangleright$ form static segments and slots
5: $T \leftarrow \langle \text{SYSTEM}(x), G, \tau \rangle$ $\triangleright$ assemble the template
6: if $\text{ROUNDTrip}(T, U)$ then $\triangleright$ verify exact reconstruction
7: return T
8: end if
9: return $\perp$

```
- 

**3.1.2 Identifying heterogeneous evolution pattern.**

The heterogeneity identification involves Intra-callsite evolution pattern and Cross-callsites sharing. Some components may remain static across invocations, some are shared across call sites, some are accumulating as execution progresses, and some can just be transient. The identification comes from history analysis.

**Intra-callsite evolution.** For each binding, we compare its value across consecutive calls of the same callsite within a session. We and count same, extend, and fresh transitions. We record the transitions as *same*, *extend*, and *fresh*. Concretely, *same* means binding values remains identical, *extend* means the new value extends the old value by appending content to its end, and *fresh* stands for distinct values.

**Cross-callsite sharing.** Cross-callsite sharing means a binding's value is used by multiple callsites. We check whether another callsite in the same session already carried the same value, or a prefix that the current value extends. Sharing expose an extra opportunity of making a prefix reusable across callsites.

We characterize each binding by one of four evolution behaviors:

- *Constant*: the same literal recurs across sessions;
- *Copied*: the binding retains or reuses a value observed earlier in the same session;
- *Extended*: the new value extends an earlier value by appending;
- *Volatile*: the binding is usually replaced by a fresh value.

For each binding name  $n$  at callsite  $x$ , we record both its value origin and its evolution between consecutive invocations of  $x$ . An origin rule is considered dominant after it has been observed at least twice and accounts for at least half of the binding's observations.

A binding is *Constant* when its dominant origin is a recurring literal. Otherwise, when same-callsite transitions exist,

let

$$\rho(x, n) = \frac{N_{\text{fresh}}}{N_{\text{same}} + N_{\text{extend}} + N_{\text{fresh}}}.$$

Conceptually, the binding is *Volatile* when  $\rho(x, n) > \frac{1}{2}$ ; otherwise, its non-fresh behavior is *Copied* or *Extended*, according to whether  $N_{\text{same}}$  or  $N_{\text{extend}}$  is larger.

A callsite invoked only once per session provides no same-callsite transitions. In that case, its dominant origin determines whether the binding is copied or extended; an unresolved or other origin is treated as volatile for stability ordering.

Independently, a binding is marked *cross-stable* when:

1. at least one observation is shared from a different callsite;
2. cross-callsite sharing accounts for at least half of its observations; and
3. either no same-callsite transitions exist or  $\rho(x, n) \leq \frac{1}{2}$ .

We represent the ordering stability of binding  $n$  at callsite  $x$  by

$$\sigma(x, n) = (c(x, n), \rho(x, n), r(x, n)),$$

where

$$c(x, n) = \begin{cases} 0, & \text{if } (x, n) \text{ is cross-stable,} \\ 1, & \text{otherwise.} \end{cases}$$

For a constant binding, we set

$$(\rho(x, n), r(x, n)) = (-1, 0).$$

When transition observations exist,  $\rho(x, n)$  is the fresh-transition fraction, and  $r(x, n)$  is the rank of the most frequent non-fresh transition:

$$r(\text{same}) = 1, \quad r(\text{extend}) = 2,$$

with  $r = 3$  when no non-fresh transition has been observed.

When no transition observations exist, we set  $\rho(x, n) = 1$  and derive  $r(x, n)$  from the dominant origin:

$$r(\text{copy}) = 1, r(\text{extend}) = 2, r(\text{other or unresolved}) = 3.$$

Bindings are ordered lexicographically by  $\sigma$ , so cross-stable bindings precede other bindings, followed by increasingly volatile bindings.

**3.1.3 Deriving application-level agent TPFM.** IDONT-KNOWITSNAME treats agents as time-annotated probabilistic finite-state machine(TPFM). We formulate TPFM for agent applications as follows:

$$\mathcal{A} = (Q, P, q_0, F, \mathcal{T}),$$

where:

1.  $Q$  is the set of states. Each state  $q \in Q$  is labelled by an LLM callsite;
2.  $P$  denotes the transition probabilities between states.
3.  $q_0 \in Q$  is the entrance callsite.
4.  $F$  specifies the termination probability of the agent at each state.

5.  $\mathcal{T} = \{D_q^{\text{exec}}, D_{q,q'}^{\text{gap}}\}$  denotes the timing annotations associated with states and transitions, where  $D_q^{\text{exec}}$  is the execution-time distribution of the LLM callsite represented by state  $q$ , and  $D_{q,q'}^{\text{gap}}$  is the inter-call gap distribution between the completion of callsite  $q$  and the invocation of its successor  $q'$ .

During one session, the server observes only a sequence of LLM calls. IDONTKNOWITSNAME aggregates the completed sequences into a prefix-tree acceptor whose nodes and edges retain their observed frequencies. For each application, the automaton is rebuilt whenever the number of completed sessions reaches  $8 \cdot 2^k$ , for  $k = 0, 1, 2, \dots$ . During rebuilding, IDONTKNOWITSNAME applies an Alergia-style red-blue state-merging procedure to the prefix tree.

$$\left| \frac{c_1}{n_1} - \frac{c_2}{n_2} \right| < \sqrt{\frac{1}{2} \ln \frac{2}{\alpha} \left( \frac{1}{\sqrt{n_1}} + \frac{1}{\sqrt{n_2}} \right)}, \quad \alpha = 0.05.$$

Here,  $n_i$  is the number of observed visits to state  $i$ , and  $c_i$  is the number of those visits that produce the particular outcome. Once two states are merged, their visit, termination, transition, and timing observations are combined to construct the resulting TPFM.

### 3.2 Heterogeneity-aware KV Cache scheduling

The discovery plane identifies where reuse exists: the ordering-stability tuple characterizes the lifetime and sharing scope of each binding, the value-flow rules identify which future binding values can be reconstructed, and the TPFM supplies call-transition and timing observations. The scheduling plane turns this information into cache actions by exposing reusable content as a prefix and effective KV cache placement. IDONTKNOWITSNAME combines prompt re-layout, future KV-demand estimation, deadline-driven promotion, and prediction-aware eviction to achieve the goal.

**3.2.1 Exposing reuse through prompt re-layout.** KV-cache reuse requires requests to share an identical token prefix. However, developer-defined declaration order may not account for this requirement, resulting in reduced prefix reuse. IDONTKNOWITSNAME therefore revise the binding order of each callsite according to the heterogeneity learned by the discovery plane.

For every binding  $n$  of callsite  $x$ , IDONTKNOWITSNAME uses the ordering-stability tuple  $\sigma(x, n) = (c(x, n), \rho(x, n), r(x, n))$  defined above. Once every binding at the callsite has accumulated enough observations, the bindings are stably sorted in lexicographic order of  $\sigma$  and the resulting layout is frozen. Consequently, cross-stable bindings lead the request, constants precede session-varying values, bindings with a smaller fresh-transition ratio precede more volatile bindings, and same-valued bindings precede append-only bindings

---

### Algorithm 2 Heterogeneity-aware prompt re-layout

---

**Require:** Callsite  $x$ , system prompt  $S_x$ , header  $H_x$ , bindings  $B$ , request-only tail  $\tau_x$

**Ensure:** Re-laid-out prompt

```

1: π_x : layout of callsite x
2: if π_x is not frozen then
3: if any binding has insufficient observations then
4: return $S_x \parallel H_x \parallel \text{RENDER}(B) \parallel \tau_x$
5: end if
6: $\pi_x \leftarrow \text{STABLESORT}(\text{NAMES}(B), \text{key} = \sigma(x, \cdot))$
7: Freeze π_x
8: end if
9: $V \leftarrow \text{RENDER}(B, \pi_x)$
10: if π_x is nonempty and $c(x, \pi_x(1)) = 0$ then
11: return $S_x \parallel V \parallel H_x \parallel \tau_x$
12: else
13: return $S_x \parallel H_x \parallel V \parallel \tau_x$
14: end if
```

---

when their preceding components tie. Bindings with identical ordering-stability tuples retain their original relative order.

Ordinarily, the re-laid-out request has the logical form

$$S_x \parallel H_x \parallel b_{\pi(1)} \parallel \dots \parallel b_{\pi(m)} \parallel \tau_x,$$

where  $S_x$  is the system prompt,  $H_x$  is the callsite-specific header and schema,  $\pi$  is the learned binding order, and  $\tau_x$  is the request-only tail. If the first binding is cross-stable,  $H_x$  would prevent two different callsites from sharing that binding as a prefix. In this case, IDONTKNOWITSNAME places the callsite-specific header after the binding values:

$$S_x \parallel b_{\pi(1)} \parallel \dots \parallel b_{\pi(m)} \parallel H_x \parallel \tau_x.$$

This header-last layout exposes reuse across callsites without changing the system prompt or the binding contents.

### 3.3 Prediction-guided KV Management

Re-layout exposes reusable prefixes in incoming requests, while proactive cache placement requires anticipating future reuse. IDONTKNOWITSNAME predicts when future calls will arrive and uses binding evolution patterns to reconstruct their known prefixes, guiding fine-grained KV-cache prefetching and retention. For every live session, IDONTKNOWITSNAME maintains the observed callsite history  $h_t = (x_1, \dots, x_t)$  and periodically produces a forecast

$$\widehat{C}_t = \{(x, p_x, t_{50,x}, t_{90,x})\},$$

where  $x$  is a possible future callsite,  $p_x$  is its cumulative reachability within the planning horizon, and  $t_{50,x}$  and  $t_{90,x}$  are the estimated median and pessimistic arrival times relative to the current timing anchor.

**Single-step next-call probability estimation.** Because transition frequencies can be sparse during online learning, IDONTKNOWITSNAME complements the constructed TPFM with a bounded-context predictor over callsite sequences. The predictor stores all observed callsite subsequences of length at most  $K = 4$  in a context trie. For a context  $u$ , let  $C_u(x)$  be the number of times callsite  $x$  follows  $u$ , let  $C_u(\perp)$  be the number of sessions that terminate after  $u$ , and define

$$n_u = C_u(\perp) + \sum_x C_u(x),$$

$$T_u = |\{x : C_u(x) > 0\}| + 1[C_u(\perp) > 0].$$

The confidence assigned to context  $u$  is

$$\lambda_u = \frac{n_u}{n_u + T_u}.$$

Starting from the empty context, IDONTKNOWITSNAME applies Witten–Bell-style interpolation over increasingly long observed suffixes:

$$\hat{P}(x | u) = \lambda_u \frac{C_u(x)}{n_u} + (1 - \lambda_u) \hat{P}(x | \text{suffix}(u)),$$

$$\hat{P}(\perp | u) = \lambda_u \frac{C_u(\perp)}{n_u} + (1 - \lambda_u) \hat{P}(\perp | \text{suffix}(u)).$$

A well-supported long context can dominate the prediction, whereas a sparsely observed context is interpolated with its better-supported shorter suffixes rather than being used as a deterministic prediction. The termination outcome only consumes probability mass but won't create a successor in the prediction tree.

**Cumulative reachability and arrival time.** A single next-call prediction is insufficient for preparing calls several steps ahead. IDONTKNOWITSNAME therefore performs a bounded best-first expansion of possible future executions. We define frontier element as:

$$(\pi, h, q, u_{50}, s^2, d),$$

where  $\pi$  is the partial-path probability,  $h$  is its hypothetical callsite history,  $q$  is its current TPFM state,  $u_{50}$  is the estimated median completion time of its last call,  $s^2$  is its accumulated squared timing spread, and  $d$  is its depth. The initial frontier contains  $(1, h_t, q_t, 0, 0, 0)$ .

For a candidate successor  $x$ , the new path probability is  $\pi' = \pi \hat{P}(x | h)$ . Let  $q'$  be the state reached by  $x$ , and let

$$g_{50} = Q_{0.5}(D_{q,q'}^{\text{gap}}), \quad g_{90} = Q_{0.9}(D_{q,q'}^{\text{gap}}).$$

The estimated arrival times and spread of  $x$  are

$$a_{50} = u_{50} + g_{50}, \quad s_a^2 = s^2 + (g_{90} - g_{50})^2, \quad a_{90} = a_{50} + \sqrt{s_a^2}.$$

To expand the path beyond  $x$ , IDONTKNOWITSNAME adds the execution time of  $x$ . With

$$e_{50} = Q_{0.5}(D_{q'}^{\text{exec}}), \quad e_{90} = Q_{0.9}(D_{q'}^{\text{exec}}),$$

the successor frontier element becomes

$$(\pi', h \parallel x, q', a_{50} + e_{50}, s_a^2 + (e_{90} - e_{50})^2, d + 1).$$

The search expands the highest-probability frontier first and retains at most the three most likely successors of each frontier element. It prunes a path when its probability falls below  $p_{\min} = 0.02$ , median arrival exceeds the 120-second planning horizon, depth exceeds eight, or the search has expanded 64 frontier elements. When multiple paths reach the same callsite  $x$ , their path probabilities are accumulated and capped at one,

$$p_x = \min \left( 1, \sum_{\varrho: \varrho \leadsto x} P(\varrho) \right),$$

while the earliest predicted arrival pair is retained for scheduling that callsite.

**3.3.1 Prediction-guided KV Management.** IDONTKNOWITSNAME performs proactive KV cache scheduling guided by heterogeneity and prediction. Deadline-driven KV promotion and Prediction-aware eviction are involved in the KV cache management.

**Deadline-driven KV promotion.** Deadline-driven KV promotion means anticipating the incoming prompt before it arrives, and value flow rules obtained via 3.1 helps. Constants are inserted directly; copied values and values taken from earlier bindings and responses are resolved from the current session; and an extended value contributes the known prefix of its accumulated representation. Such reconstruction process proceeds by bindings, and it stops at the first unresolved binding. For every predicted call, IDONTKNOWITSNAME chooses the longest cache target and uses statically known callsite prefix as a conservative fallback.

After selecting the scheduled KV cache, IDONTKNOWITSNAME placed the promotion target with estimated deadline. For a predicted call  $c = (x, p_x, t_{50,x}, t_{90,x})$  and timing anchor  $a$ , IDONTKNOWITSNAME assigns the deadline

$$d_c = \max(t_{\text{now}}, a + t_{50,x} - \kappa(t_{90,x} - t_{50,x})), \quad \kappa = 0.5.$$

The term  $\kappa(t_{90} - t_{50})$  is to moves uncertain jobs earlier and let scheduler to prepare the prefix more conservatively.

Also, the absolute pessimistic arrival time  $\bar{t}_c = \max(d_c, a + t_{90})$  is for garbage collection. If the predicted call has not arrived after  $\bar{t}_c$ , its cache-plan entry is considered stale and discarded.

Let  $U_c$  be the number of target-prefix tokens absent from both cache tiers, let  $H_c$  be the number resident in host memory but absent from device memory, and let  $R_{\text{prefill}}$  and  $R_{\text{promote}}$  be the profiled prefill-under-load and host-to-device transfer rates. The latest safe preparation time is estimated as

$$r_c = d_c - \frac{U_c}{R_{\text{prefill}}} - \frac{H_c}{R_{\text{promote}}}.$$

A promotion job becomes eligible whenever the engine is idle or the current time approaches to  $r_c$ . Among eligible jobs, the scheduler selects the earliest deadline. During promotion, active kvcache and ongoing tasks will be protected to ensure that speculative preparation won't interfere serving requests.

**Prediction-aware eviction.** IDONTKNOWITSNAME exploits both predicted usage and prompt heterogeneity in KV eviction scheduling. Device-resident radix nodes are divided into four lifecycle-aware priority bands, listed by the order of eviction priority sequence:

1. *retired*: private tail nodes belonging to completed sessions;
2. *transient*: non-reconstructible prompt tails and generated descendants that are unlikely to be reused;
3. *unclassified*: ordinary cache not covered by the current plan;
4. *planned*: nodes required by one or more active predictions.

The scheduler uses LRU order within the *retired*, *transient*, and *unclassified* bands. For KV cache nodes associated with *Planned* band, they are sorted by aggregated protection score and then by recency.

Band transitions occur at three events. After a request completes, nodes beyond its longest pre-arrival reconstructible prefix are marked *transient*. When the session terminates, its *transient* tails become *retired*. At every plan refresh, IDONTKNOWITSNAME removes the previous planned protections before applying the new scores, preventing stale predictions from retaining priority.

The host tier then serves as a recover layer. It evicts retired entries first and otherwise follows LRU. A prediction error will be less likely to turn into a full recomputation.

Additionally, IDONTKNOWITSNAME converts every live prediction into an eviction-protection score:

$$S_c(t) = p_x \exp\left(-\frac{\max(0, d_c - t)}{\tau}\right), \quad \tau = 60 \text{ s.}$$

This score is added to the eviction priority of every radix-tree node on the prefix's path. Nodes with *Planned* band are evicted in increasing order of accumulated score, then LRU.

## 4 Evaluation

### 4.1 Enviromental Setup

**Configurations.** We evaluated IDONTKNOWITSNAME on a single RTX-3090 with Qwen-8B(bf16). IDONTKNOWITSNAME is implemented on SGLang with HiCache with chunked-prefill [1] enabled. For Baseline comparison, we selected vanilla SGLang and two opeaqe requests serving systems CacheScout and Continuum [8, 11].

#### Workloads.

**Workloads.** We evaluate IDONTKNOWITSNAME using two representative agent applications: a fact-checking agent and a coding agent. Using these applications, we execute two benchmarks: HoVer [6], a multi-hop fact verification benchmark requiring evidence collection and reasoning across

---

### Algorithm 3 Prediction-aware eviction

---

**Require:** Cache tier  $C$ , required free space  $M$ , live predictions  $\mathcal{P}$ , current time  $t$

**Require:** Lifecycle labels updated on request completion and session termination

**Ensure:** Whether at least  $M$  free space is available

```

1: while FREESPACE(C) < M do
2: $E \leftarrow \text{EVICTABLELEAVES}(C)$ $\triangleright$ Exclude locked nodes
3: if $E = \emptyset$ then
4: return false
5: end if
6: for all $v \in E$ do
7: $s_v \leftarrow 0$
8: if v is retired then
9: $b_v \leftarrow 0$
10: else if v is transient then
11: $b_v \leftarrow 1$
12: else
13: $b_v \leftarrow 2$ $\triangleright$ Unclassified
14: end if
15: if C is the device tier then
16: $\mathcal{P}_v \leftarrow \{c \in \mathcal{P} : v \in \text{PATH}(c)\}$
17: if $\mathcal{P}_v \neq \emptyset$ then
18: $b_v \leftarrow 3$ $\triangleright$ Planned
19: $s_v \leftarrow \sum_{c \in \mathcal{P}_v} S_c(t)$
20: end if
21: else
22: $b_v \leftarrow \min(b_v, 1)$ $\triangleright$ Host: retired first, then
23: end if
24: $K_v \leftarrow (b_v, s_v, \text{LASTACCESS}(v))$
25: end for
26: $v^* \leftarrow \arg \min_{v \in E} K_v$ $\triangleright$ Lexicographic order
27: $\text{EVICT}(C, v^*)$
28: end while
29: return true

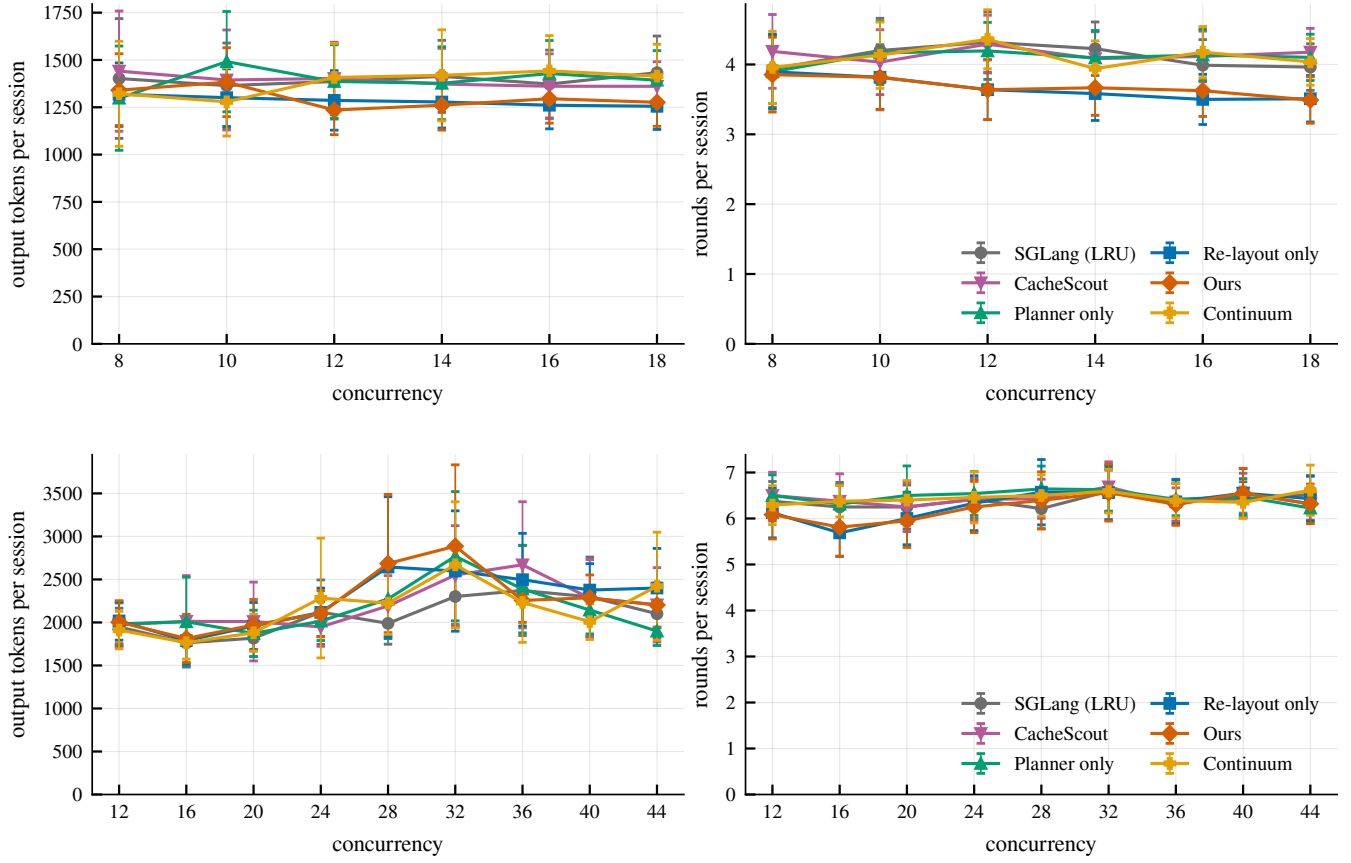
```

---

multiple Wikipedia articles, and HumanEval [2], a code-generation benchmark that evaluates the functional correctness of generated programs using unit tests.

### 4.2 Task quality and execution scale

Because IDONTKNOWITSNAME introduces prompt re-lay out, which would change the ordering inside prompt. Such transformation should keep comparable task quality and scale. As shown in Figure 2 and Table 1. As shown in the table, all configurations achieve comparable task accuracy. At the same time, output tokens per session remains essentially same. The observed serving improvements do not come at the cost of output quality and reduction of task scale.



**Figure 2.** Application-level task quality under varying concurrency for the HoVer fact-checking agent (left) and the HumanEval coding agent (right).

**Table 1.** Application-level task quality across serving configurations.

| Method         | Fact Checking | Code Generation |
|----------------|---------------|-----------------|
| SGLang (LRU)   | 57.2%         | 47.6%           |
| CacheScout     | 57.2%         | 47.2%           |
| Planner only   | 54.9%         | 46.2%           |
| Re-layout only | 56.4%         | 45.5%           |
| <b>Ours</b>    | <b>58.0%</b>  | <b>45.3%</b>    |

### 4.3 Evaluation Analysis

Our evaluations on IDONTKNOWITSNAME address the following research questions:

1. How much does IDONTKNOWITSNAME improve end-to-end serving performance?
2. How much does two scheduling mechanisms of IDONTKNOWITSNAME contribute to the improvement?
3. How do the two mechanisms of heterogeneity-aware KV-cache scheduling in IDONTKNOWITSNAME interact and reinforce each other?

**4.3.1 RQ1: How IDONTKNOWITSNAME improve end-to-end performance.** We evaluate IDONTKNOWITSNAME on two representative agent applications: a fact-checking agent on the HoVer dataset and a coding agent on the HumanEval dataset. We use mean job completion time (JCT) and mean time to first token (TTFT) as the end-to-end performance metrics.

**JCT.** As shown in Figures 4a, 4b, 3b, and 3a, IDONTKNOWITSNAME consistently reduces mean JCT over the vanilla SGLang baseline, achieving  $1.3\times$ – $1.7\times$  speedup on the fact-check agent and  $1.1\times$ – $1.3\times$  on the coding agent. On the fact-check agent, Continuum performs comparably to SGLang at low concurrency but its advantage vanishes as load increases, whereas IDONTKNOWITSNAME sustains its gains across all concurrency levels. On the coding agent, IDONTKNOWITSNAME outperforms every other system; CacheScout and Continuum degrade sharply at high concurrency and fall behind SGLang.

**TTFT.** IDONTKNOWITSNAME also lowers mean TTFT across all workload configurations (Figures 4c and 4d). On the fact-check agent, the reduction is substantial at every concurrency level, while Continuum and CacheScout offer little or no improvement over SGLang. On the coding agent, IDONTKNOWITSNAME matches or beats all baselines up to a concurrency of 40; at the highest concurrency, KV-only achieves a slightly lower TTFT.

#### 4.3.2 RQ2: How much two scheduling mechanisms of IDONTKNOWITSNAME contribute to the improvement.

The improvement of IDONTKNOWITSNAME comes from more efficient KV cache scheduling. From figure 5, it can be cleared observed that performance gain comes from lower KV cache miss rate. IDONTKNOWITSNAME’s Heterogeneity-aware KV Cache scheduling helps keeping useful KV entries in cache. This scheduling mechanism incorporates two aspect, heterogeneity aware prompt re-layout and Prediction-guided KV Management. We thus investigate into detailed serving status to understand how the two kv scheduling mechanisms contributes to performance improvement.

**Speed-up introduced by heterogeneity-aware prompt re-layout.** To investigate the effects of prompt re-layout, We conduct an ablation experiment which disables IDONTKNOWITSNAME’s KV scheduling mechanism, and only use heterogeneity-aware prompt re-layout, as shown in the *re-layout only* arm in 5. In serving code agent, which is a decode heavy agent, re-layout gains nearly all optimization when concurrency is less than 28. But when workload continues to grow, re-layout itself cannot sustain useful KV, cache miss rate begins to drop, leading to larger gap between it and full IDONTKNOWITSNAME. Across sessions of fact check agent, re-layout only keeps a steady slight more cache miss rate, which caused a performance deficiency compared to full IDONTKNOWITSNAME.

These evaluation results clearly demonstrate that heterogeneity aware prompt re-layout itself helps enhancing performance. Across all workload pressure, heterogeneity-aware prompt re-layout improves performance by introducing more opportunities of kv sharing, thus reduced cache miss and optimizes serving performance. It could even achieve full performance IDONTKNOWITSNAME in some cases, but is still behind full IDONTKNOWITSNAME under workload pressure.

**Effects of Prediction-guided KV Management.** Adopting similar methodology, we create an ablation arm that only activate KV management of IDONTKNOWITSNAME, which is the KV-only arm in figure 5. This mechanism shows a substantially distinct behavior with prompt re-layout. In fact check agent serving sessions, it only works under low pressure, where  $c \leq 14$ , which is very similar to other kv management frameworks: Continuum and CacheScout also work only under very low concurrency level. Moreover, this

arm, together with Continuum and CacheScout, even stay inert in serving code agent.

The reason for this phenomenon is quite straightforward: KV cache management needs space to operate. Prediction-guided KV Management’s effects come from keeping useful entries in cache, prevent them from being evicted. However, when pressure is high, ongoing requests competes for memory, none kv cache can survive. Consequently, KV cache management’s effect diminishes under high workload pressure. Continuum and CacheScout can also be observed with similar phenomenon. But effect of Prediction-guided KV Management of IDONTKNOWITSNAME disappear much later than Continuum and CacheScout. In fact check agent, Continuum and CacheScout lose their functionality at  $c = 10$ , while IDONTKNOWITSNAME’s Prediction-guided KV Management survives to  $c = 14$ .

#### 4.3.3 RQ3: How two mechanisms in IDONTKNOWITSNAME’s KV scheduling interact and reinforce each other.

We analyze the two mechanism’s interactions and co-reinforcement, we scrutinize kv cache statistics and investigate into some representative pressure points to figure out. We first analyze the behavior pattern of IDONTKNOWITSNAME under high workload.

In serving code agent, *re-layout only* obtains same performance as full IDONTKNOWITSNAME. But under high workload pressure, full IDONTKNOWITSNAME obviously works better. Enabling IDONTKNOWITSNAME’s kv cache management mechanism furthermore improves serving performance. Comparing the cache status in figure 5a, *re-layout only* keeps same cache miss rate initially(When  $c \leq 28$ ) but device miss increases. Host tier catches the mistakenly evicted kv entries. *re-layout only* has same cache miss rate but higher device miss rate. The efficient host-to-device kv cache loading algorithms in SGLang fill the gap, therefore performance doesn’t diverge at those points. When concurrency grows to more than 32, full IDONTKNOWITSNAME begins outperforming *re-layout only*. It has less cache miss, leading to better performance.

This behavior also emerges when testing on the fact check agent. As we mentioned above, IDONTKNOWITSNAME’s kv management algorithm only works under low pressure, where  $c \leq 14$ . Heterogeneity-aware prompt re-layout has effect across all pressure because it creates more kv sharing opportunities, which leaves space for kv scheduling. Therefore, heterogeneity-aware prompt re-layout and prediction-guided KV management are not isolated two mechanisms, prompt re-layout create spaces, and kv management further improves cache efficiency.

We selected some representative workload pressure and investigate how many tokens are recomputed. We choose  $c = 12, 28, 40$  for code agent and  $c = 8, 14, 18$  for fact check agent. As shown in 6a, prompt re-layout maintains exactly the same recomputed tokens as full IDONTKNOWITSNAME,

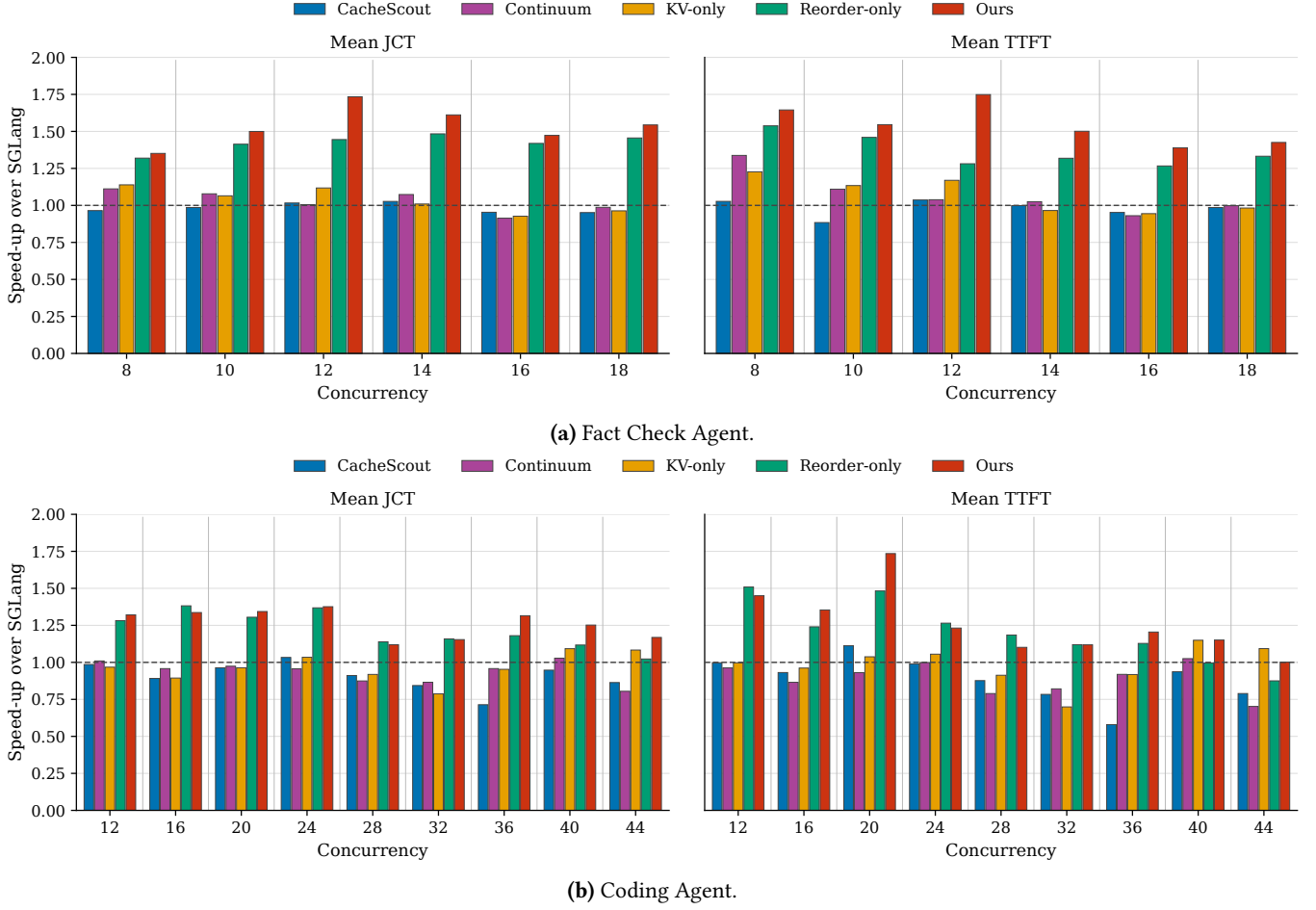


Figure 3. Speed-up over SGLang in mean JCT and mean TTFT across concurrency levels.

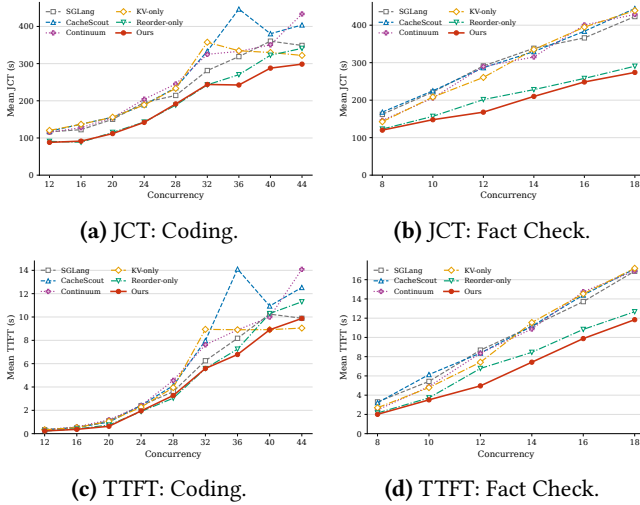
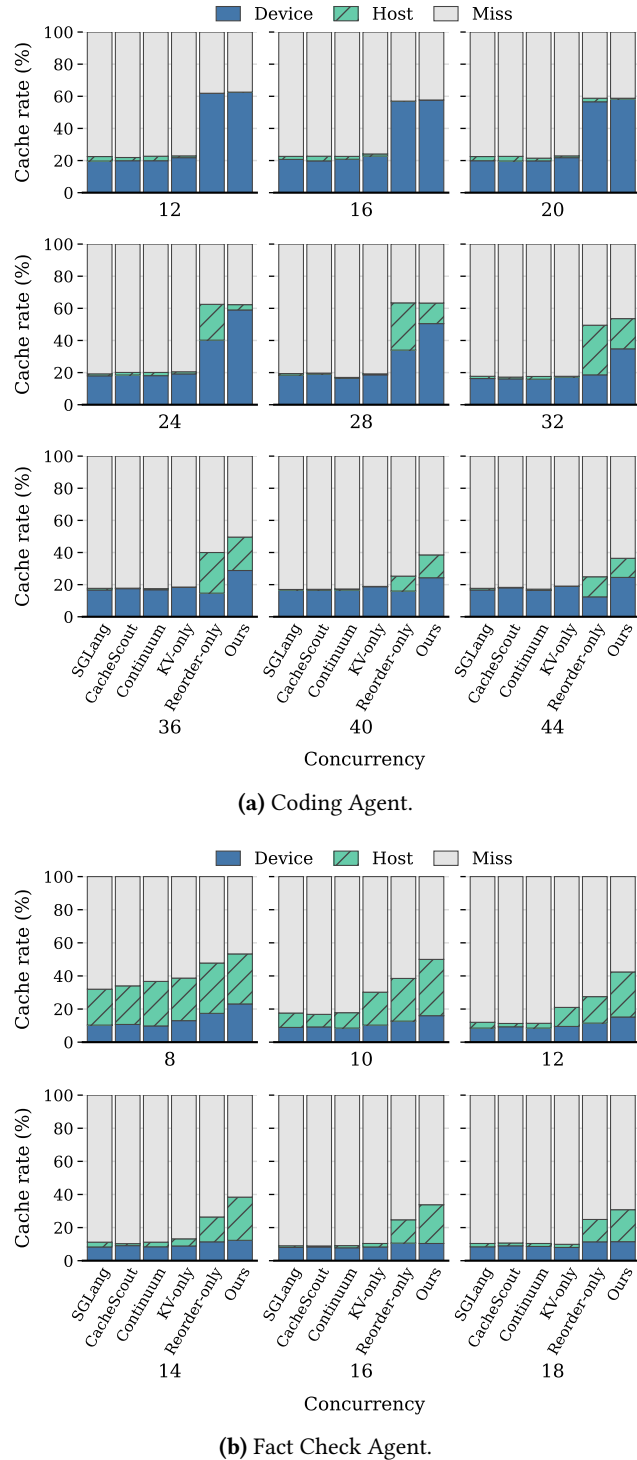


Figure 4. Mean JCT and TTFT across concurrency levels for the Coding Agent and Fact Check Agent.

but for  $c=40$ , things are different, re-layout only is left behind, adding kv scheduling reduces tokens recomputed furthermore. In the regime of fact agent, all kv scheduling mechanisms reduce the recomputed tokens. CacheScout, Continuum and IDONTKNOWITSNAME all enhance cache efficiency. Similarly, when workload is high, IDONTKNOWITSNAME's prediction-guided KV management could add optimization upon the heterogeneity aware prompt re-layout, but kv scheduling itself cannot take effect.

## References

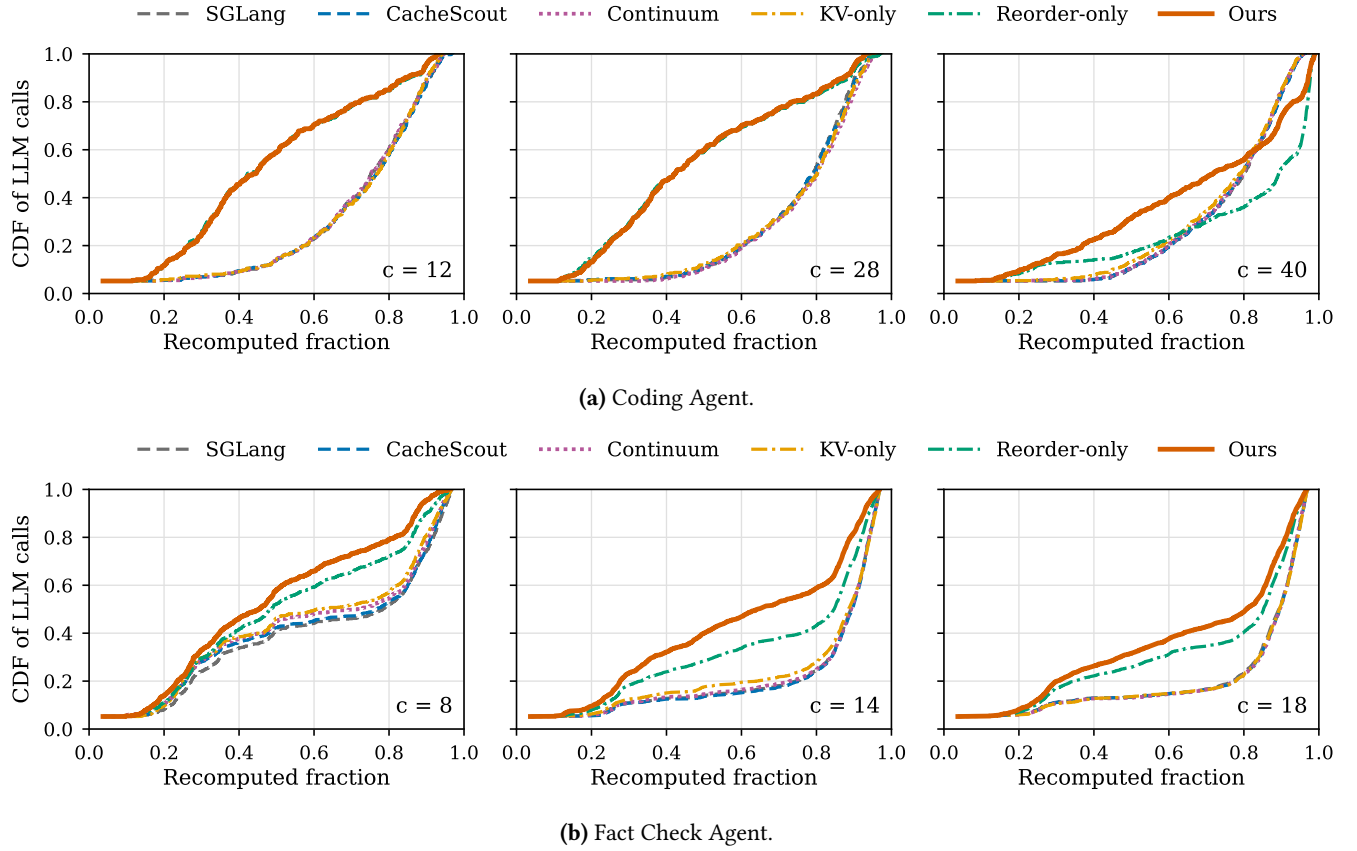
- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 117–134.
- [2] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet,



**Figure 5.** End-to-end KV-cache status (device hit, host hit, miss) under different concurrency levels.

Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgens Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin,

- Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).
- [3] Jayanaka L. Dantanarayana, Yiping Kang, Kugesan Sivasothyathan, Christopher Clarke, Baichuan Li, Savini Kashmira, Krisztian Flautner, Lingjia Tang, and Jason Mars. 2025. MTP: A Meaning-Typed Language Abstraction for AI-Integrated Programming. *Proceedings of the ACM on Programming Languages* 9, OOPSLA2 (2025), 1176–1204. doi:10.1145/3763092
- [4] Jayanaka L. Dantanarayana, Savini Kashmira, Lingjia Tang, and Jason Mars. 2026. SIGIL: Compiling Agent Skills into Typed Harnesses. arXiv:2607.27309 [cs.SE] <https://arxiv.org/abs/2607.27309>
- [5] Paul Furgale, Severin Klingler, James Nolan, Matt Staats, Gaia Di Lorenzo, Elisa Martinez Abad, Christian Schüller, Razvan Dinu, Alessio Devoto, Pascal Berard, Gal Kaplun, Elad Sarafian, Riccardo Roveri, Leon Derczynski, and Ricardo Silveira Cabral. 2026. NVIDIA-labs OO Agents: Native Python Object-Oriented Agents. arXiv:2607.20709 [cs.AI] <https://arxiv.org/abs/2607.20709>
- [6] Yichen Jiang, Shikha Bordia, Zheng Zhong, Charles Dognin, Maneesh Singh, and Mohit Bansal. 2020. HoVer: A Dataset for Many-Hop Fact Extraction And Claim Verification. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. 3441–3460.
- [7] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines. In *International Conference on Learning Representations*.
- [8] Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Huanchen Zhang, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. 2025. Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live. arXiv:2511.02230 [cs.OS]
- [9] Zaifeng Pan, Ajikumar Patel, Zhengding Hu, Yipeng Shen, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. 2025. KVFlow: Efficient Prefix Caching for Accelerating LLM-Based Multi-Agent Workflows. arXiv:2507.07400 [cs.DC] <https://arxiv.org/abs/2507.07400>
- [10] Shan Yu, Junyi Shu, Yuanjiang Ni, Kun Qian, Xue Li, Yang Wang, Jinyuan Zhang, Ziyi Xu, Shuo Yang, Lingjun Zhu, Ennan Zhai, Qingda Lu, Jiarong Xing, Youyou Lu, Xin Jin, Xuanzhe Liu, and Harry Xu. 2026. Pythia: Exploiting Workflow Predictability for Efficient Agent-Native LLM Serving. arXiv:2604.25899 [cs.MA] <https://arxiv.org/abs/2604.25899>
- [11] Rui Zhang, Chaeun Kim, Shaoting Feng, Kuntai Du, Yuhuan Liu, Yi Zhong, Cheng-Wei Ching, Junchen Jiang, and Liting Hu. 2026. Learning Agent Execution for KV-Cache Management in Agentic Serving. arXiv:2608.14624 [cs.AI]
- [12] Haoyu Zheng, Fangcheng Fu, Jia Wu, Binhang Yuan, Yongqiang Zhang, Hao Wang, Yuanyuan Zhu, Xiao Yan, and Jiawei Jiang. 2026. Efficient Serving for Dynamic Agent Workflows with Prediction-based KV-Cache Management. arXiv:2605.06472 [cs.LG] <https://arxiv.org/abs/2605.06472>
- [13] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems*, Vol. 37.



**Figure 6.** CDF of the recomputed-token fraction per LLM call at representative concurrency levels.